Accumulii: Dev Hub

A Web Development 1 Proposal
Presented to the Faculty of the
Department of Computer, Information Sciences and Mathematics
University of San Carlos

In Partial Fulfillment
of the Requirements for the
WEB DEVELOPMENT 1

By
Abaya, Joshua Danielle Ermac
Campus, John Louis
Elopre, Joshua Reed Omamalin
Tallo, Lance Benedict

CHRIS RAY B. BELARMINO
Instructor

May 2026

Table of Contents

Table of Contents....2
CHAPTER 1....1
INTRODUCTION....1
1.1 Rationale....1
1.2 Objectives....2
1.2.1 General Objectives....2
1.2.3 Specific Objectives....2
1.3 Beneficiaries....2
1.4 Scope and Limitation....3
CHAPTER 2....4
ANALYSIS AND DESIGN....4
2.1 Business Process....4
2.2 Users....4
2.3 ERD....4
2.4 Tools Used....4
2.5 Functional Requirements....4
2.6 Non-functional Requirements....4
REFERENCES....5
APPENDICES....6
Appendix A....7
Appendix B....7

CHAPTER 1

INTRODUCTION

1.1 Rationale

Most coding platforms today are all about the work, such as hosting code and tracking changes. But they often ignore a huge part of developer culture: customizing your workspace. Developers and designers spend a lot of time tweaking their terminals, writing custom scripts, and making their desktops look awesome. While communities like "unixporn" exist, there hasn't been a dedicated app that mixes professional networking with this kind of creative expression.

Accumulii Dev Hub was built to fix this. Think of it as a social hub just for programmers. It gives users a place to show off their custom setups (*rices*), post updates on what they are building (*devlogs*), and share coding tutorials. By giving it a retro CRT and terminal look, the app feels like it truly belongs to the developer community, standing out from standard, boring work tools.

1.2 Objectives

1.2.1 General Objectives

The general objective of Accumulii is to create an integrated platform that facilitates collaboration, learning, and presentation of development projects within a developer-focused ecosystem. .

1.2.3 Specific Objectives

To make this happen, we need to hit these specific goals:

1. To provide a platform for showcasing developer configurations and projects
2. To enable users to publish and browse development logs (devlogs)
3. To support structured write-ups for technical documentation and architecture discussions
4. To allow users to explore and connect with other developers in the network
5. To implement an interactive and user-friendly interface with search and filtering capabilities

1.3 Beneficiaries

Accumulii Dev Hub is designed to help a few key groups:

- **Developers and Programmers –** for sharing projects, workflows, and technical knowledge
- **Students (IT / Web Development) –** for learning through real-world examples and write-ups
- **Designers and Engineers –** for showcasing creative and technical outputs
- **Tech Communities –** for collaboration and knowledge exchange

1.4 Scope and Limitation

Scope

The system includes modules such as About Us, Showcases, Devlogs, Write-ups, and Developer Profiles. It provides features like search functionality, categorized filtering *(recent, popular)*, interactive dashboards, and media viewers. The platform supports content browsing, developer interaction, and project visualization.

Limitation

This app isn't meant to replace GitHub or run code directly; it will just link out to actual code repositories. For now, it will only be a web app *(no native iOS or Android versions)*. Also, we will only host images and text for the posts—users won't be able to upload heavy software files or compiled programs directly to the site.

CHAPTER 2

ANALYSIS AND DESIGN

2.1 Business Process

Accumulii Dev Hub operates through a terminal-driven web interface that simulates a Unix-like command-line environment. The main flow of the platform is as follows:

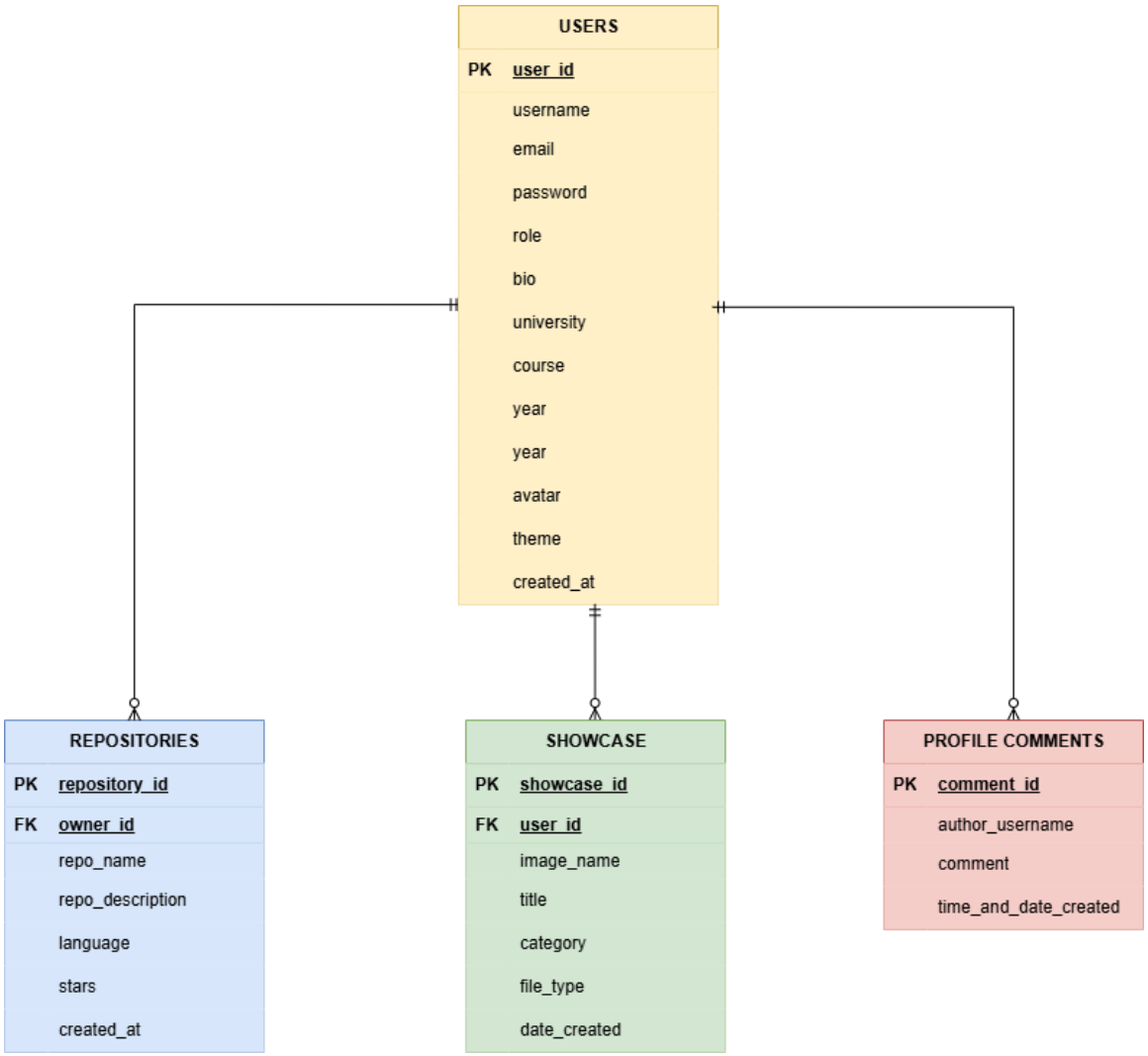
- **Authentication -** Before using any features, users must create an account and log in. All API endpoints reject unauthenticated requests with an unauthorized error, and the system uses PHP sessions to validate credentials server-side.
- **Profile Management -** Once logged in, users can view their own or another developer's profile using the profile <username> command. Profile data includes username, role, bio, university, course, and year. Users may update editable fields (bio, university, course, year) using the set <key> "value" command, or reset all profile data to defaults via restore defts.
- **Repository Showcase -** Users can list all repositories in the registry using repos, search by keyword with searchrepo <query>, and inspect one or more repository details using repo <name>. Repositories are linked to user accounts and display metadata such as programming language and star count.
- **Visual Showcases -** Developers can browse desktop environment screenshots and UI setups (rices) through the showcases command, and view a specific showcase in full resolution with showcase <name>. Each showcase belongs to a user and is categorized (e.g., Ricing, Terminal).
- **System Broadcast Log -** Any authenticated user can post a public message to the system broadcast log using post "<message>", and read the global log using the log command. This serves as an informal, community-wide announcement board.
- **Theme Customization -** Users can switch the terminal's visual theme between dark, ash, and white using the theme <name> command. The preference is persisted in the database per user.

2.2 Users

As of right now, Accumulii Dev Hub only supports one user role: Developer. Every registered user has the same level of site access and capabilities. The present implementation does not have an admin panel or moderator position.

User Type	Description	Access Level
Guest	Unauthenticated visitor.	None. will be redirected to login.
Registered Developer	Any user with a registered account.	Full access to all terminal commands and features.

2.3 ERD



The database is built on **MySQL** and consists of four tables:

USERS — stores account credentials, profile information (bio, university, course, year), avatar, and theme preference.

REPOSITORIES — stores repository metadata (name, description, language, stars) linked to a user via user_id.

SHOWCASES — stores image showcase entries (title, category, file type) linked to a user via user_id.

PROFILE_COMMENTS — stores public broadcast log messages associated with a username.

The ERD is shown above. Key relationships: one user can own many repositories, upload many showcases, and post many broadcast messages.

2.4 Tools Used

Based on the system specifications and architecture outlined in the proposal, the following tools and technologies are utilized for development:

- **Front-End Technologies:** HTML, CSS, and JavaScript to construct the "terminal-driven web interface that simulates a Unix-like command-line environment".

- **Back-End Language:** PHP, specifically utilized to handle server-side logic and manage "PHP sessions to validate credentials server-side".

- **Database Management System:** MySQL, which is used to build the database encompassing the users, repositories, showcases, and profile comments tables.

- **Modeling Tool:** diagrams.net (formerly Draw.io), utilized for designing the Entity Relationship Diagram (ERD).

2.5 Functional Requirements

The system must satisfy the following functional requirements to support the intended business processes:

- **Authentication:** The system must allow users to create an account, log in, and securely validate credentials, rejecting any unauthenticated API requests.

- **Profile Management:** The system must allow authenticated users to view developer profiles using the profile <username> command. It must also allow users to update their bio, university, course, and year using the set <key> "value" command, or reset this data using the restore defts command.

- **Repository Integration:** The system must enable users to list all available repositories using the repos command, search for specific repositories using searchrepo <query>, and view detailed metadata using the repo <name> command.

- **Showcase Browsing:** The system must allow developers to browse categorized UI setups and desktop environments via the showcases command, and expand specific entries to full resolution using the showcase <name> command.

- **Global Broadcasting:** The system must allow any authenticated user to publish a public message to the system broadcast log via the post "<message>" command, and allow all users to read this global log using the log command.

- **Theme Customization:** The system must allow users to change the visual theme of their terminal environment to dark, ash, or white using the theme <name> command, and successfully persist this preference in the database.

2.6 Non-functional Requirements

The following non-functional requirements define the quality standards and operational characteristics of Accumulii Dev Hub.

- **Performance**
  The system must provide fast and responsive command execution within the terminal interface under normal usage conditions. Repository searches, showcase loading, and profile viewing should execute without significant delay.

- **Security**
  The system must implement secure authentication using PHP session management. Unauthorized users must not gain access to protected features or API endpoints. User credentials and account information must be securely stored within the database.

- **Usability**
  The interface must remain intuitive and easy to navigate despite the terminal-driven design. Commands should follow a consistent structure and provide understandable outputs and error messages to users.

- **Compatibility**
  The web application must function properly on commonly used desktop browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge.

- **Scalability**
  The system architecture must support future expansion, including the addition of more user roles, advanced social features, mobile responsiveness, and larger volumes of repositories and showcases without major redesign of the platform.

REFERENCES

1. MDN Web Docs
   Mozilla Developer Network. (2025). *Introduction to web development*. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Learn

2. PHP Official Documentation
   PHP Documentation Group. (2025). *PHP sessions and authentication*. PHP.net. https://www.php.net/manual/en/book.session.php

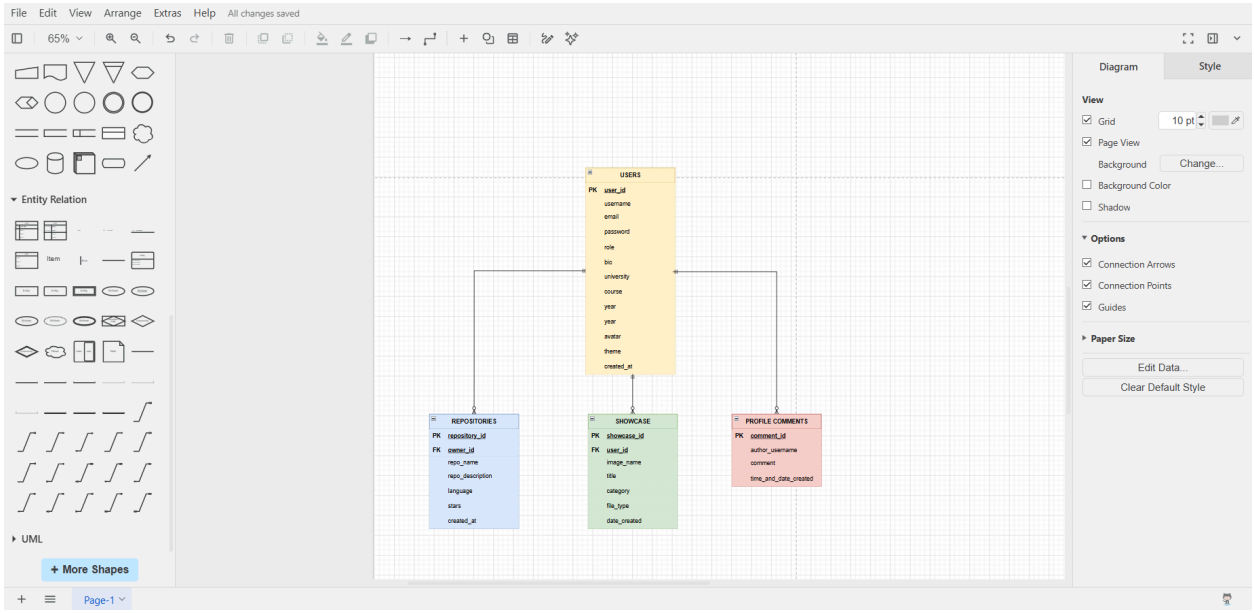
3. MySQL Documentation
   Oracle Corporation. (2025). *MySQL 8.0 Reference Manual*. Oracle. https://dev.mysql.com/doc/

4. Nielsen Norman Group
   Nielsen, J. (2020). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

5. freeCodeCamp
   Ihechikara, A. (2023). *What is an API and how does it work?* freeCodeCamp. https://www.freecodecamp.org/news/what-is-an-api-and-how-does-it-work/

Appendix A

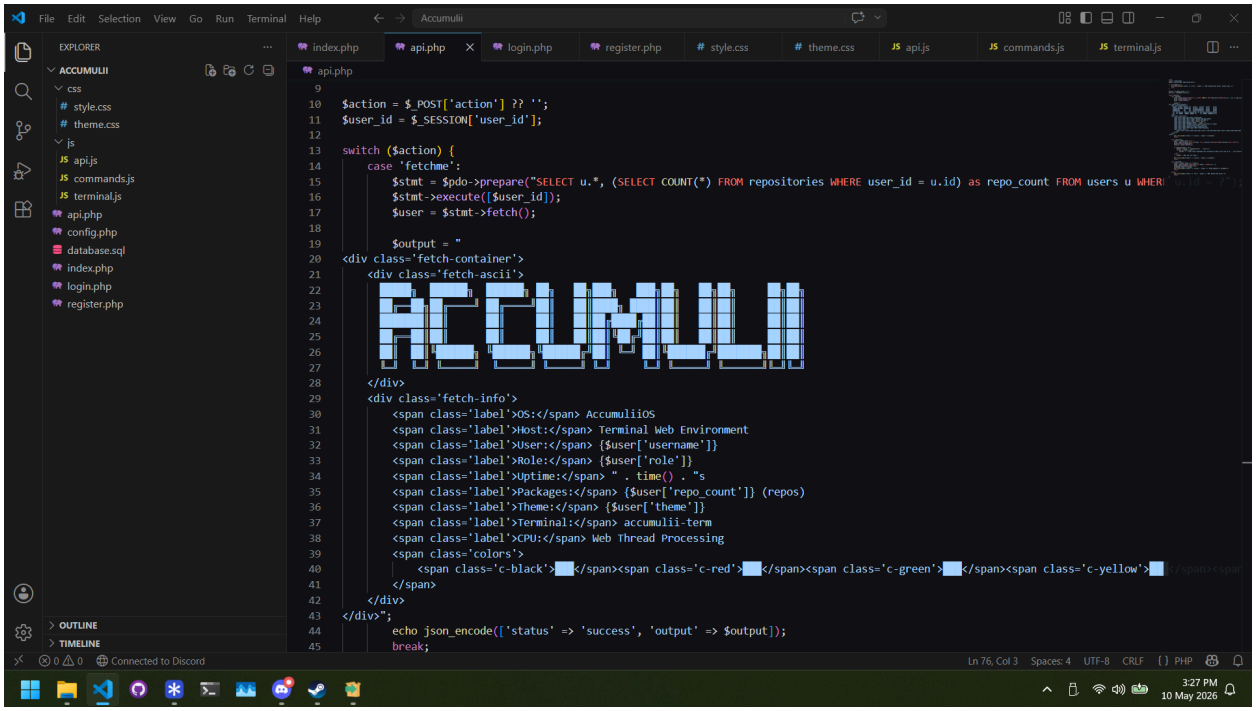
Entity Relationship Diagram (ERD) Development



This appendix presents the Entity Relationship Diagram (ERD) used in the development of Accumulii Dev Hub. The following diagrams illustrate the relationships between the database tables and entities implemented in the system.

Appendix B

Source Code




```php
$action = $_POST['action'] ?? '';
$user_id = $_SESSION['user_id'];

switch ($action) {
    case 'fetchme':
        $stmt = $pdo->prepare("SELECT u.*, (SELECT COUNT(*) FROM repositories WHERE user_id = u.id) as repo_count FROM users u WHERE
        $stmt->execute([$user_id]);
        $user = $stmt->fetch();

        $output = "
<div class='fetch-container'>
    <div class='fetch-ascii'>
    </div>
    <div class='fetch-info'>
        <span class='label'>OS:</span> AccumuliiOS
        <span class='label'>Host:</span> Terminal Web Environment
        <span class='label'>User:</span> {$user['username']}
        <span class='label'>Role:</span> {$user['role']}
        <span class='label'>Uptime:</span> " . time() . "s
        <span class='label'>Packages:</span> {$user['repo_count']} (repos)
        <span class='label'>Theme:</span> {$user['theme']}
        <span class='label'>Terminal:</span> accumulii-term
        <span class='label'>CPU:</span> Web Thread Processing
        <span class='colors'>
            <span class='c-black'>
        </span>
    </div>
</div>";
        echo json_encode(['status' => 'success', 'output' => $output]);
        break;
```

This appendix presents selected screenshot(s) of the source code and development environment used in the creation of Accumulii Dev Hub. The screenshot(s) highlight the implementation of core system functionalities such as user authentication, terminal command processing, profile management, repository browsing, and database connectivity. The project was developed using HTML, CSS, JavaScript, PHP, and MySQL within the Visual Studio Code environment. These code segments demonstrate how the application's terminal-driven interface and backend processes were structured and integrated to support the overall functionality of the system.

Appendix C

Folder Structure

```
Accumulii/
├── css/
│   ├── style.css          # Core terminal layout, design tokens, and all component styles
│   └── theme.css          # Theme overrides (dark, ash, white) via CSS custom properties
├── js/
│   ├── api.js             # Fetch wrapper — sends POST requests to api.php
│   └── commands.js        # Command parser, all command handlers, and repo keybind logic
├── img/
│   ├── profiles/          # User profile images (named by username, e.g. joshuareed.jpg)
│   └── showcases/         # Showcase images referenced by the showcases table
├── index.php              # Main terminal interface and keyboard input loop
├── login.php              # Authentication gate with animated boot sequence
├── register.php           # New user registration with multi-step prompt flow
├── api.php                # REST-style POST endpoint handling all terminal commands
├── config.php             # PDO database connection, session bootstrap, and auth helpers
└── database.sql           # Full schema with seed data (users, repos, showcases)
```

This appendix presents selected screenshots of the source code and development environment used in the creation of Accumulii Dev Hub. The screenshots highlight the implementation of core system functionalities such as user authentication, terminal command processing, profile management, repository browsing, and database connectivity. The project was developed using HTML, CSS, JavaScript, PHP, and MySQL within the Visual Studio Code environment. These code segments demonstrate how the application's terminal-driven interface and backend processes were structured and integrated to support the overall functionality of the system.